Computer Olympic POSN KU Center

Topic : Trees

Tanaboon Tongbuasirilai
Department of Computer Science
Kasetsart University

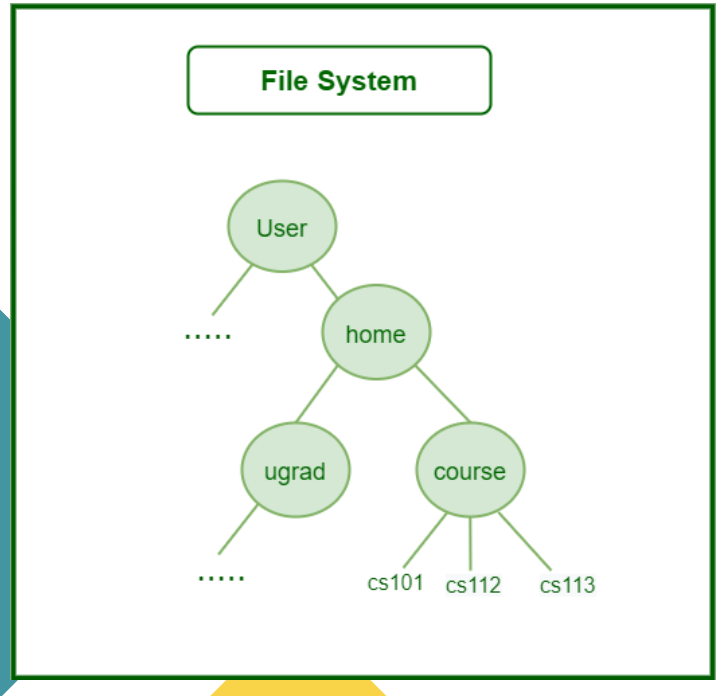
Agenda

- Introduction
- Terminology
- Types of trees
- Tree operations
- Tree traversals
- Binary tree, operations and traversals

Reference

- https://www.geeksforgeeks.org/introduction-to-tree-data-structure-and-algorithm-tutorials
- https://www.simplilearn.com/tutorials/data-structure-tutorial/trees-in-data-structure
- https://d3gt.com/unit.html?rooted-trees
- https://www.cs.umd.edu/class/fall2020/cmsc420-0201/Lects/lect03-trees.pdf
- https://www.freecodecamp.org/news/binary-search-tree-traversal-inorder-preorder-post-order-for-bst/

Introduction to Tree

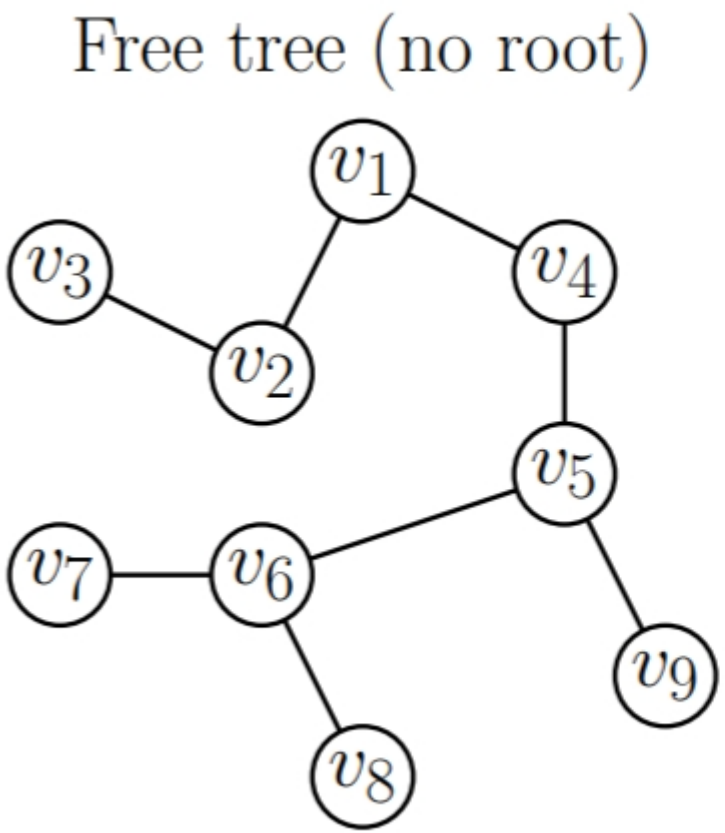


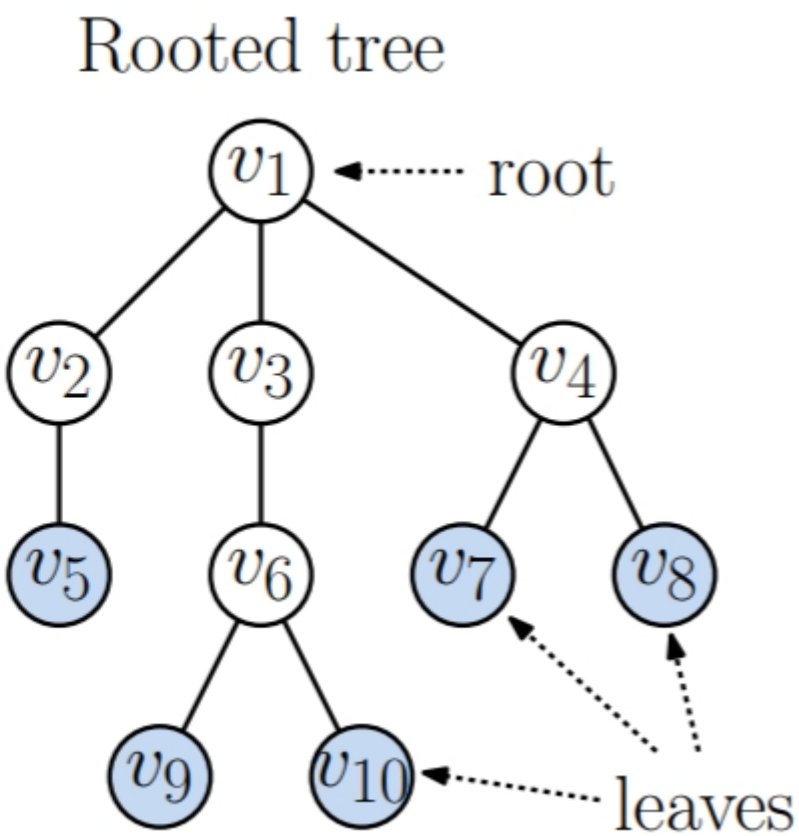
- A tree data structure is a **hierarchical structure** that is used to represent and organize data in a way that is **easy to navigate and search.** It is **a collection of nodes** that are **connected by edges** and has a hierarchical relationship between the nodes.
- The topmost node of the tree is called the **root**, and the nodes below it are called the **child nodes.** Each node can have multiple child nodes, and these child nodes can also have their own child nodes, forming a recursive structure.
- Trees are non-linear
- Trees also provide a natural organization for file systems, GUI, databases, websites, etc.

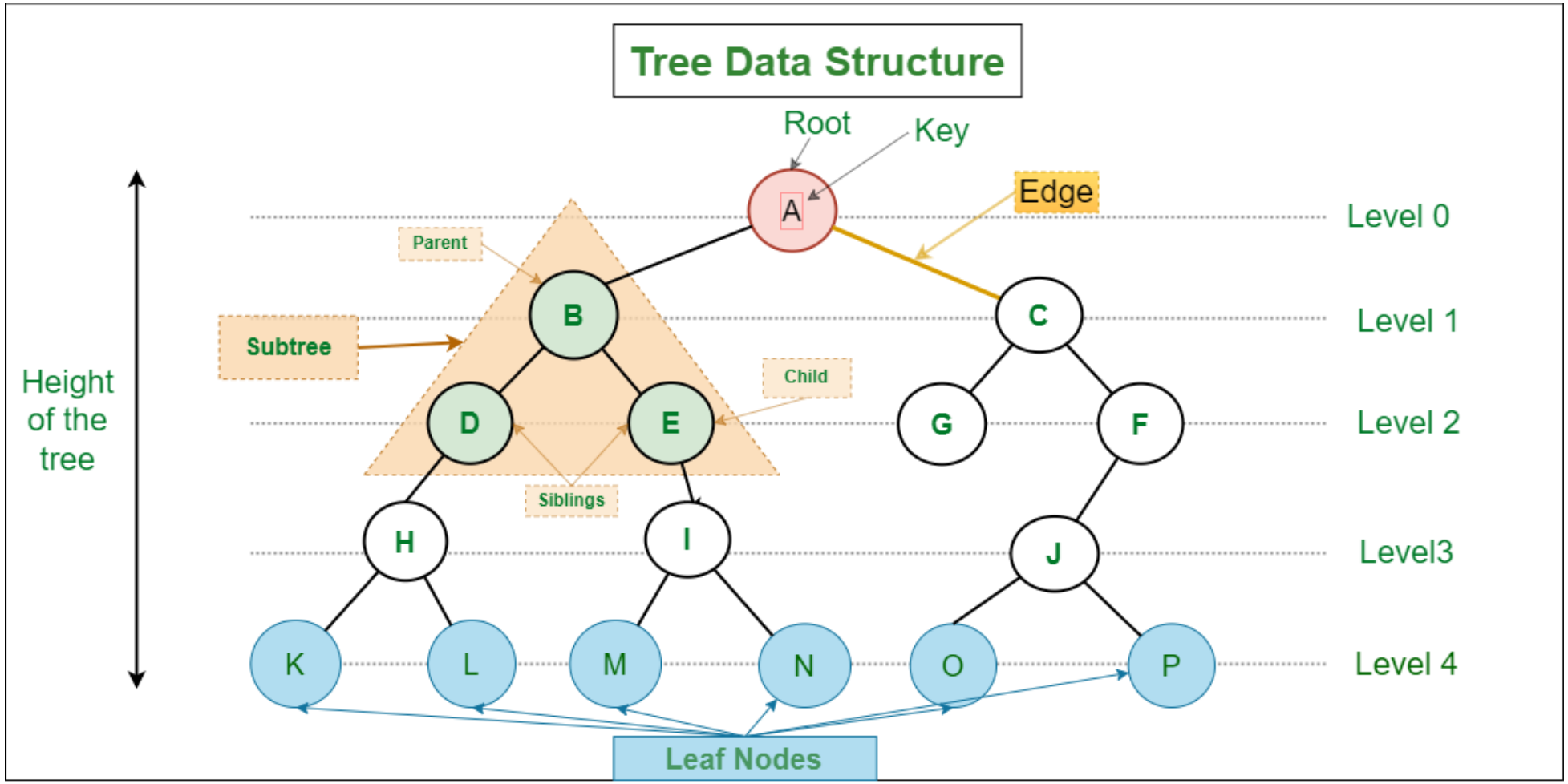
Forms of a tree

Free tree : a free tree, is simply a connected, undirected graph that has no cycles.

Rooted tree : A rooted tree is a free tree in which one of the vertices is distinguished from the others.

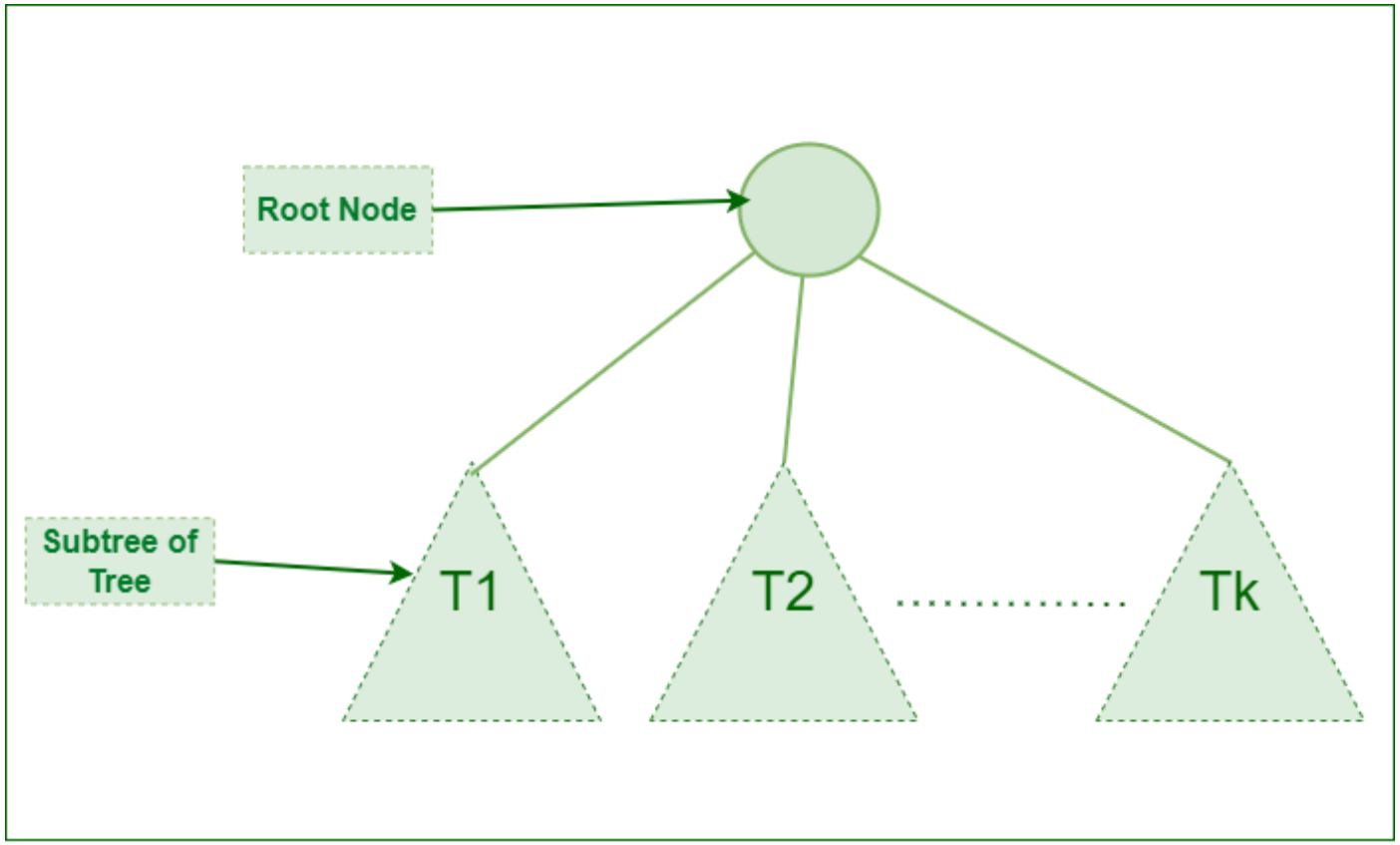


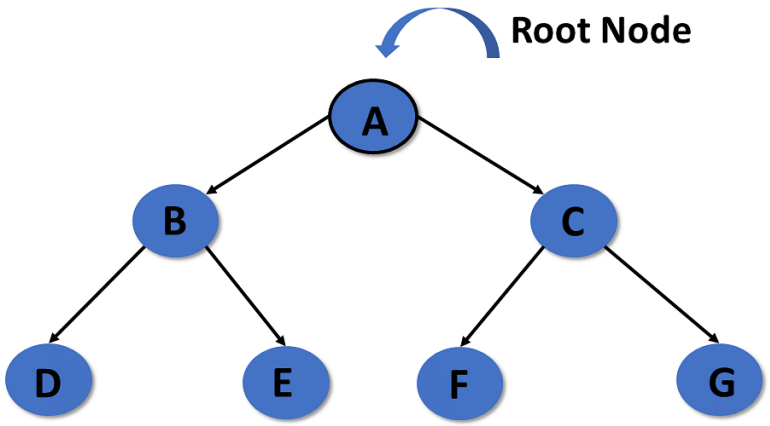


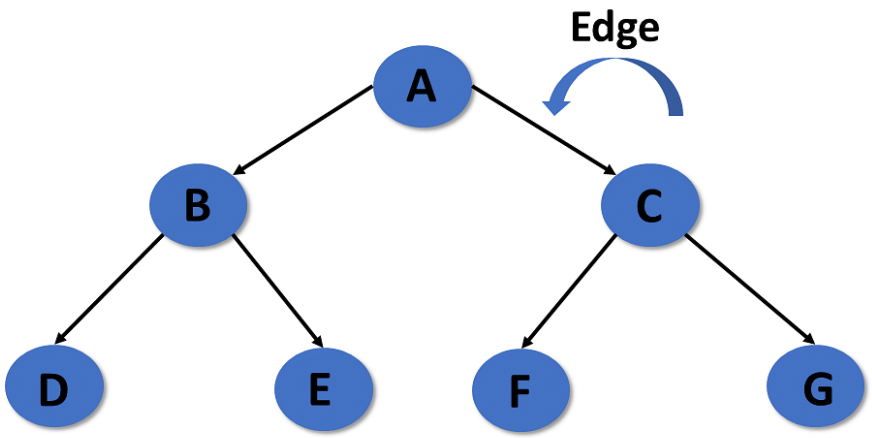


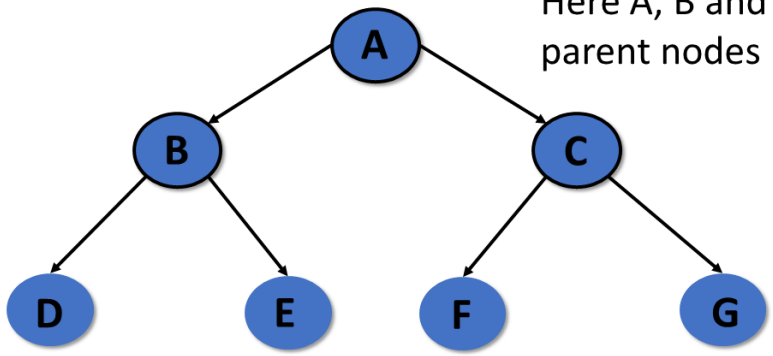
Terminology

A tree consists of a root, and zero or more subtrees $T_1, T_2, \ldots, T_k$ such that there is an edge from the root of the tree to the root of each subtree.



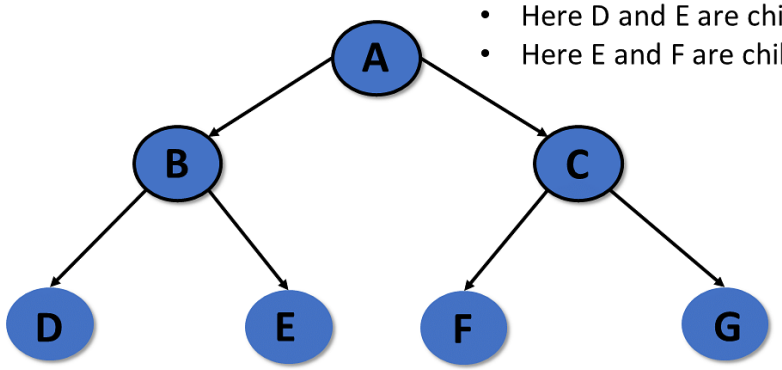




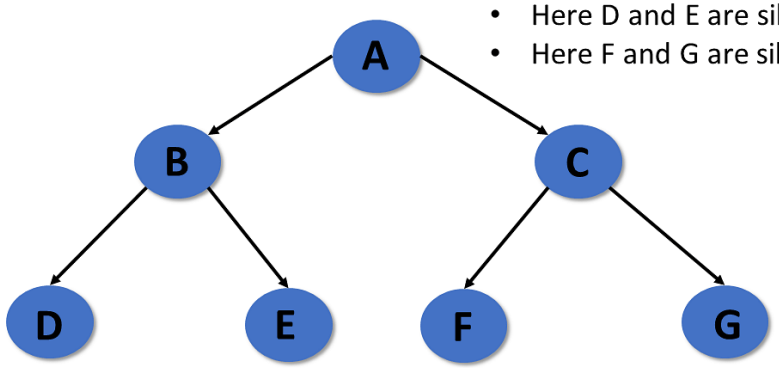


Here A, B and C are parent nodes

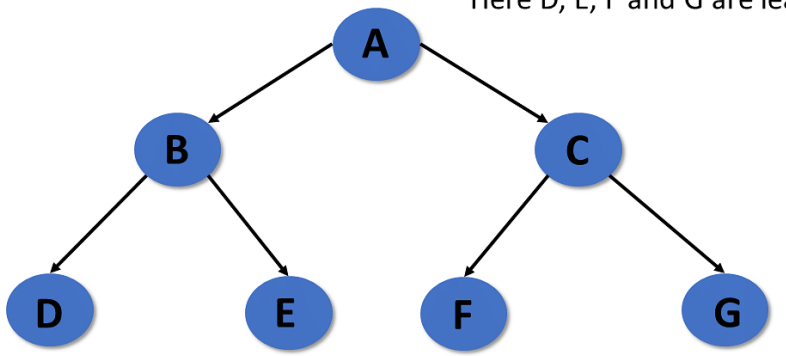
- Here B and C are children of A
- Here D and E are children of B
- Here E and F are children of C



- Here B and C are siblings
- Here D and E are siblings
- Here F and G are siblings



Here D, E, F and G are leaf nodes.



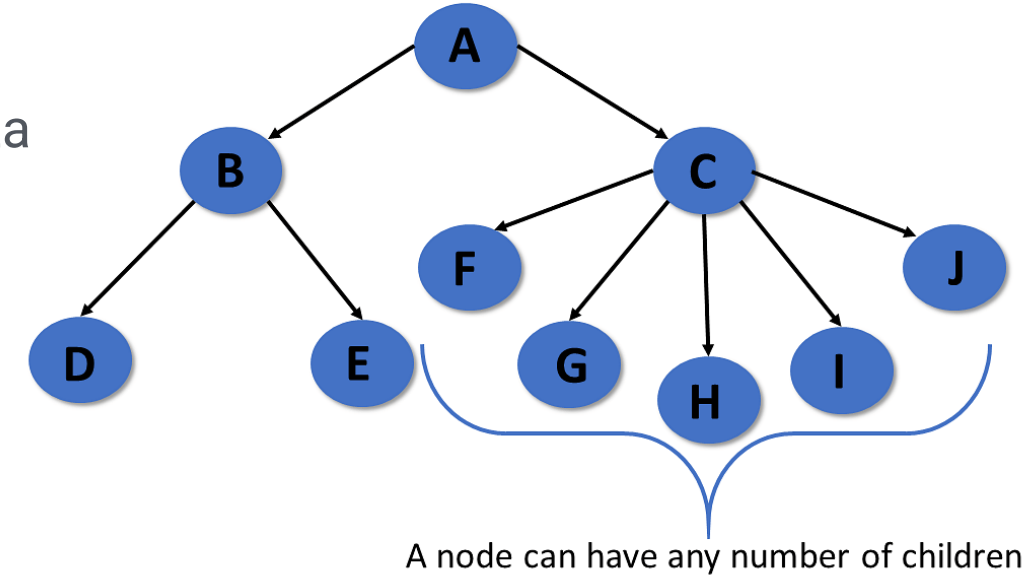
Types of trees

1. General Tree

The general tree is the type of tree where there are no constraints on the hierarchical structure.

Properties

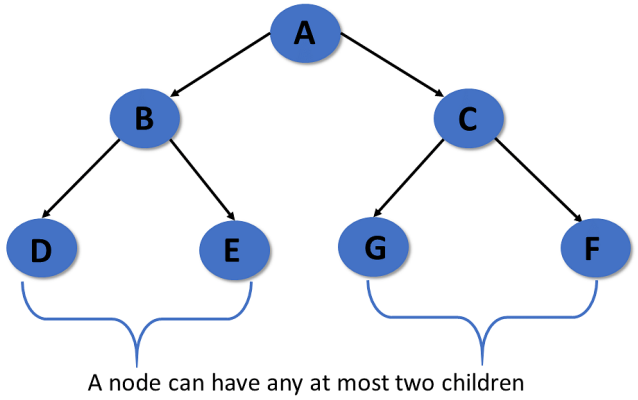
- The general tree follows all properties of the tree data structure.
- A node can have any number of nodes.



2. Binary Tree

Properties

- Follows all properties of the tree data structure.
- Binary trees can have at most two child nodes.
- These two children are called the left child and the right child.

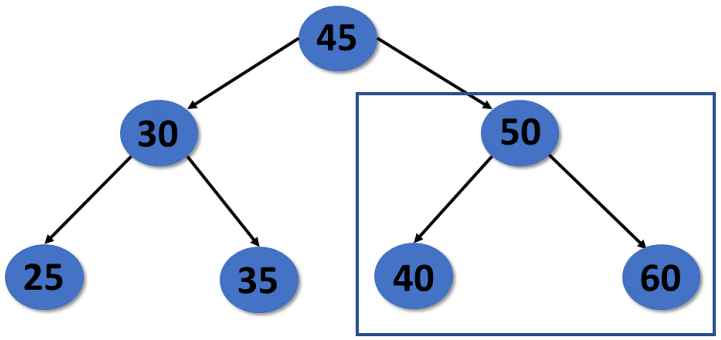


3. Binary Search Tree

A binary search tree is a type of tree that is a more constricted extension of a binary tree data structure.

Properties

- Follows all properties of the tree data structure.
- The binary search tree has a unique property known as the binary search property. This states that the value of a left child node of the tree should be less than or equal to the parent node value of the tree. And the value of the right child node should be greater than or equal to the parent value.



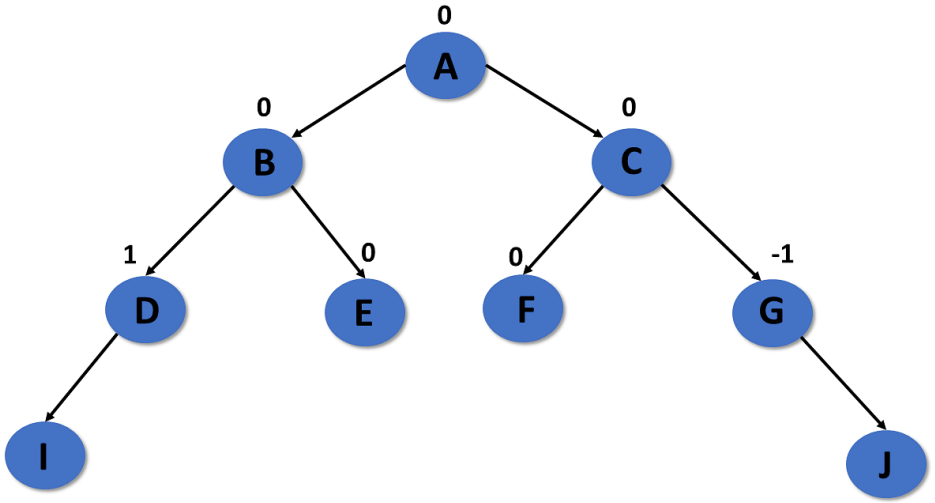
Left node value<= root node <= right node value

4. Self-Balancing Binary Search Trees

Self-Balancing Binary Search Trees are **height-balanced binary search trees** that **automatically keep the height as small as possible** when insertion and deletion operations are performed on the tree.

Properties

- Follows all properties of the tree data structure.
- Self-balancing.
- Examples: AVL Tree, Red-Black Tree, Splay Tree



Coding : Creating a rooted tree

```c
#include<stdlib.h>
struct node{
    int key;
    struct node* parent;
    struct node* leftChild;
    struct node* rightSibling;
};
```

1

```c
struct node* createNode(int key, struct node* parent) {
    // Allocate memory for new node
    struct node* newNode = (struct node*)malloc(sizeof(struct node));
    // Assign key to this node
    newNode->key = key;
    // Initialize parent
    newNode->parent = parent;
    // Initialize left child, and right sibling as NULL
    newNode->leftChild = NULL;
    newNode->rightSibling = NULL;
    return newNode;
}

struct node* insertLeft(struct node* root, int item) {
  root->leftChild = createNode(item, root);
  return root->leftChild;
}

struct node* insertRight(struct node* root, int item) {
  root->rightSibling = createNode(item, root);
  return root->rightSibling;
}
```

2

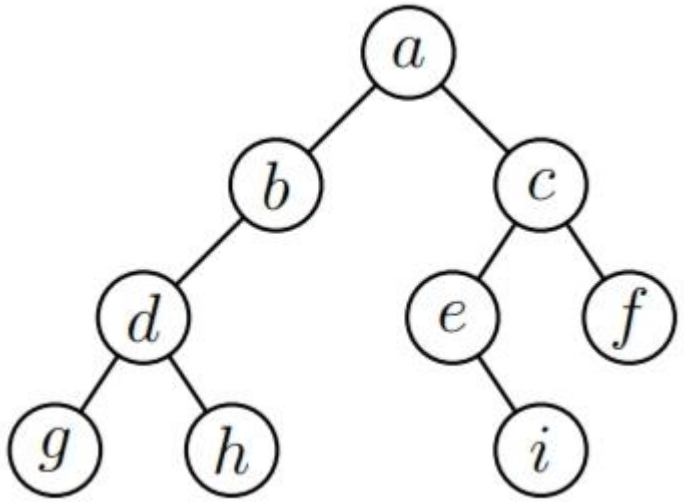
```c
int main()
{
    /*create root*/
    struct node *root = createNode(1, NULL);
    /* following is the tree after the above statement
        1
    */
    insertLeft(root, 2);
    insertRight(root, 3);
    /* 2 and 3 become children of 1
        1
      /  \
     2    3
    */
    insertLeft(root->leftChild, 4);
    /* 4 becomes left child of 2
        1
      /  \
     2    3
    /
   4
    */
    return 0;
}
```

3

Ordered trees

- An ordered tree is an oriented tree in which the children of a node are somehow ordered. It is a rooted tree in which an ordering is specified for the children of each vertex.

- A binary tree is an example of an ordered tree.



Coding : Creating an ordered tree

1

```c
struct node
{
    int key;
    struct node* parent;
    struct node* leftChild;
    struct node* rightSibling;
};
```

2

```c
struct node* createRoot(int key)
{
    // Allocate memory for new node
    struct node* node = (struct node*)malloc(sizeof(struct node));
    // Assign key to this node
    node->key = key;
    // Initialize parent
    node->parent = NULL;
    // Initialize left child, and right sibling as NULL
    node->leftChild = NULL;
    node->rightSibling = NULL;
    return(node);
}
```

3

```c
struct node* insertLeft(struct node* root, int item) {
  root->leftChild = createNode(item, root);
  return root->leftChild;
}

struct node* insertRight(struct node* root, int item) {
  root->rightSibling = createNode(item, root);
  return root->rightSibling;
}
```

A binary tree is created.

```
/*create root*/
struct node* node1 = createRoot(1);
struct node* node2 = insertLeft(node1, 2);
struct node* node3 = insertRight(node1, 3);
struct node* node4 = insertLeft(node2, 4);
/* 4 becomes left child of 2
     1
    / \
   2   3
  /
 4
*/
```

More operations

```c
void isRoot(struct node* node)
{
    if(node->parent == NULL)
        printf("Yes\n");
    else
        printf("No\n");
}
```

1

```c
void isExternal(struct node* node)
{
    if(node->leftChild == NULL)
        printf("Yes\n");
    else
        printf("No\n");
}
```

2

```c
int depth(struct node* node)
{
    int depth = 0;
    while(node->parent != NULL) {
        node = node->parent;
        depth++;
    }
    return depth;
}
```

3

```
    1
   / \
  2   3
 /
4
```

```c
isRoot(node1); // output "Yes"
isRoot(node2); // output “No”
isExternal(node2); // output "No"
isExternal(node3); // output “Yes"
printf("%d\n", depth(node4)); // output "2"
```

Complexity of operations

Operations	Complexity
createRoot()	*O(1)*
insertLeft(), insertRight()	*O(1)*
isRoot()	*O(1)*
isExternal()	*O(1)*
depth()	*O(d), d = max nodes*

Tree traversal

Unlike linear data structures (Array, Linked List, Queues, Stacks, etc) which have only one logical way to traverse them, trees can be traversed in different ways.

Traversal by Depth First Search or DFS

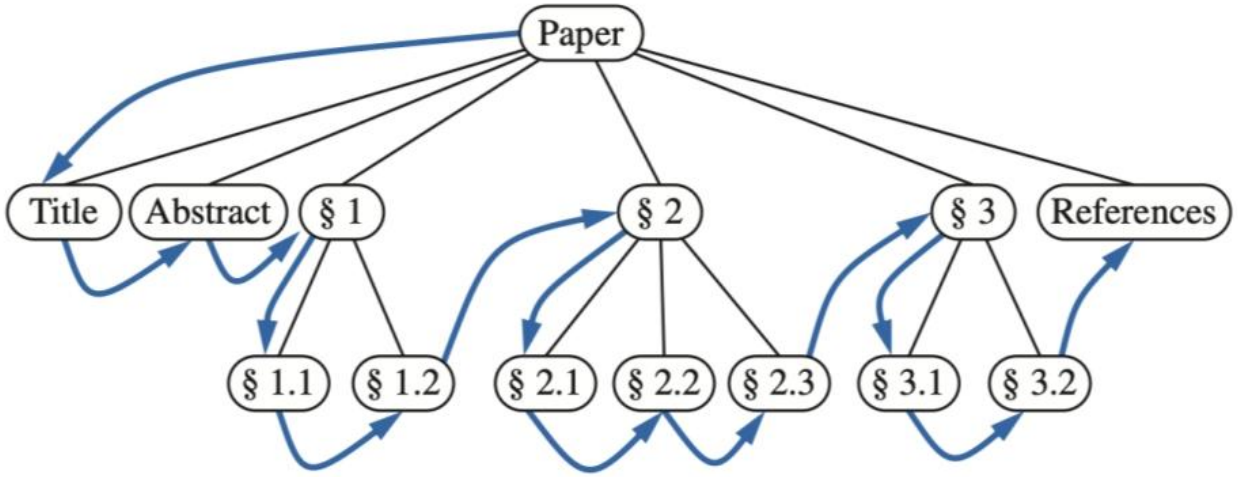
- Preorder Traversal
- Postorder Traversal
- Inorder Traversal

Traversal by Breadth First Search or BFS

- Level Order Traversal

Preorder Traversal

- In a preorder traversal of a rooted tree T, we visit the root of T first and then the subtrees rooted at its children are traversed recursively
- If the tree is ordered, then the subtrees are traversed according to the order of the children

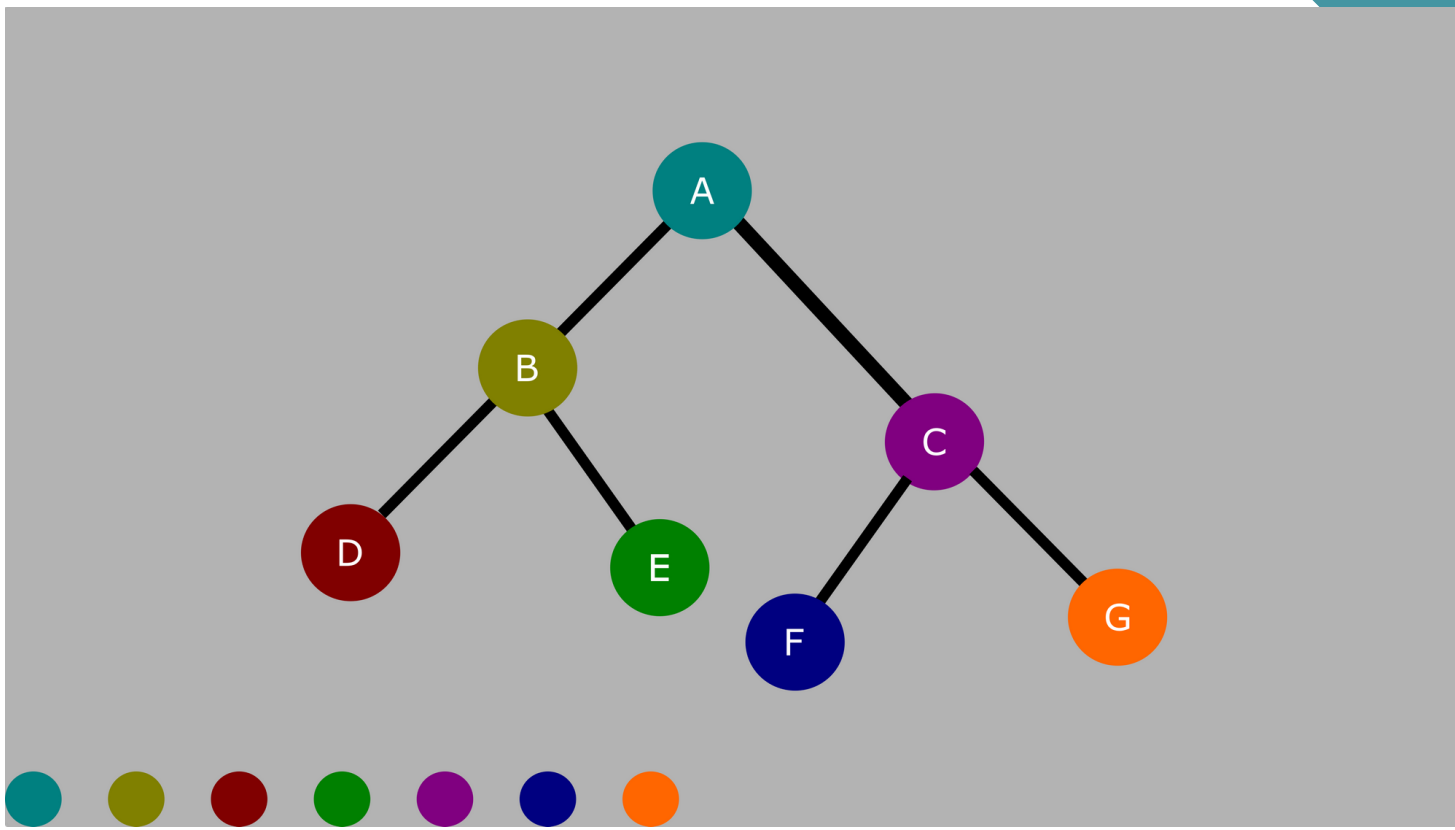


Preorder steps

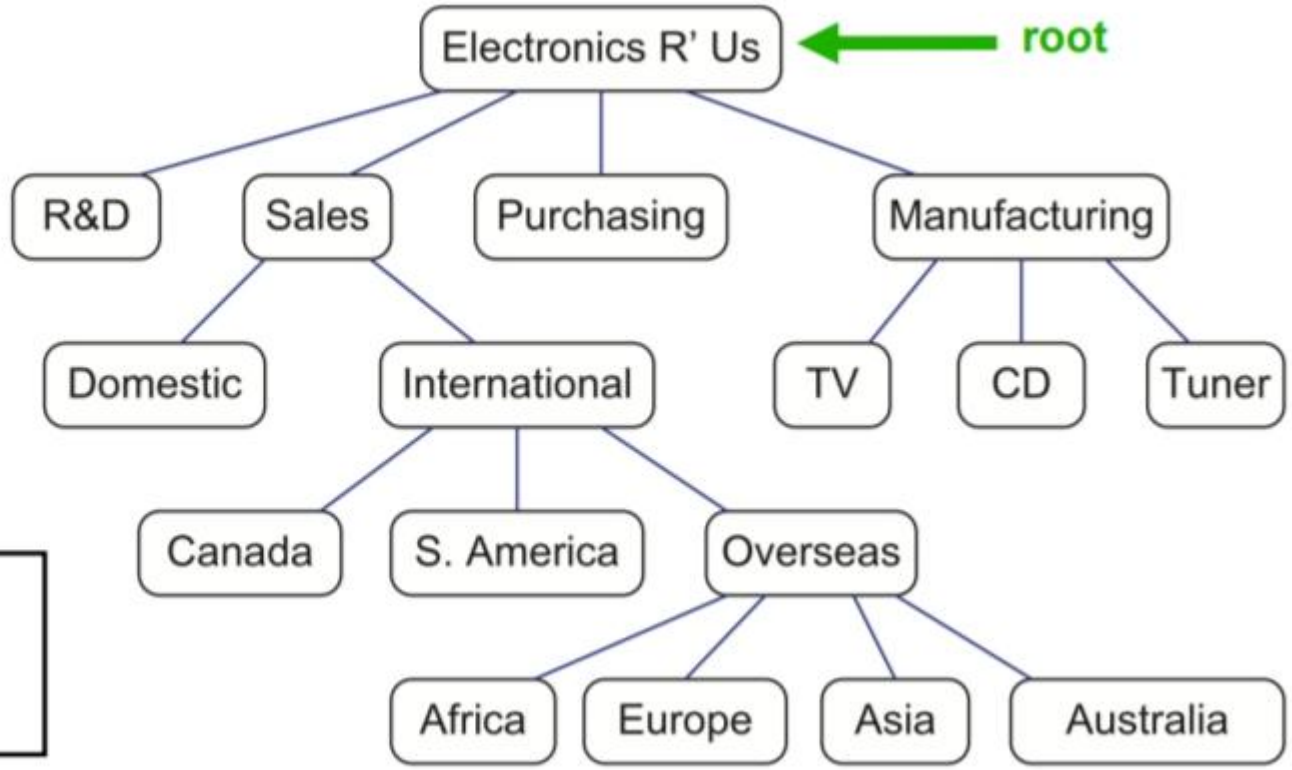
1. Visit node. (Root)
2. Go to the left subtree. (Left)
3. Go to the right subtree. (Right)

Pseudocode (Root, Left, Right)

```
preorder(r,T):
        visit node r
        for each child q of r:
                preorder(q,T)
```



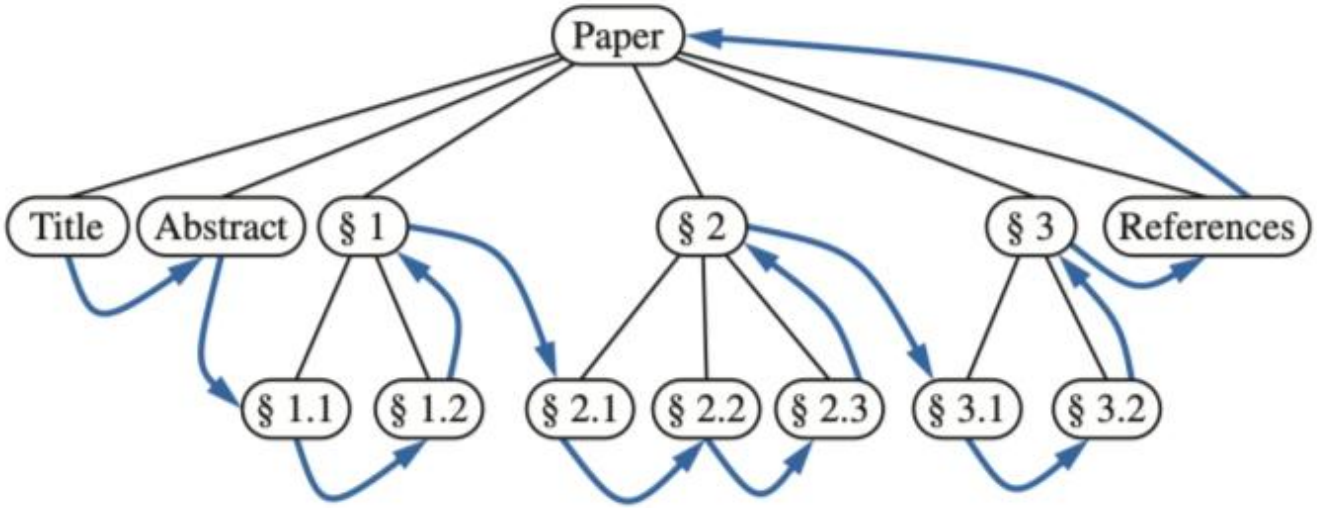
Example : Preorder traversal




```
preorder(r,T):
    visit node r
    for each child q of r:
      preorder(q,T)
```

Postorder Traversal

- Postorder traversal : As opposed to preorder traversal, in a postorder traversal of a rooted tree T, we recursively traverse the subtrees rooted at the children of the root first and then visits the root.
- In a post traversal of a rooted tree T, we recursively traverse the subtrees rooted at the children of the root first and then visits the root

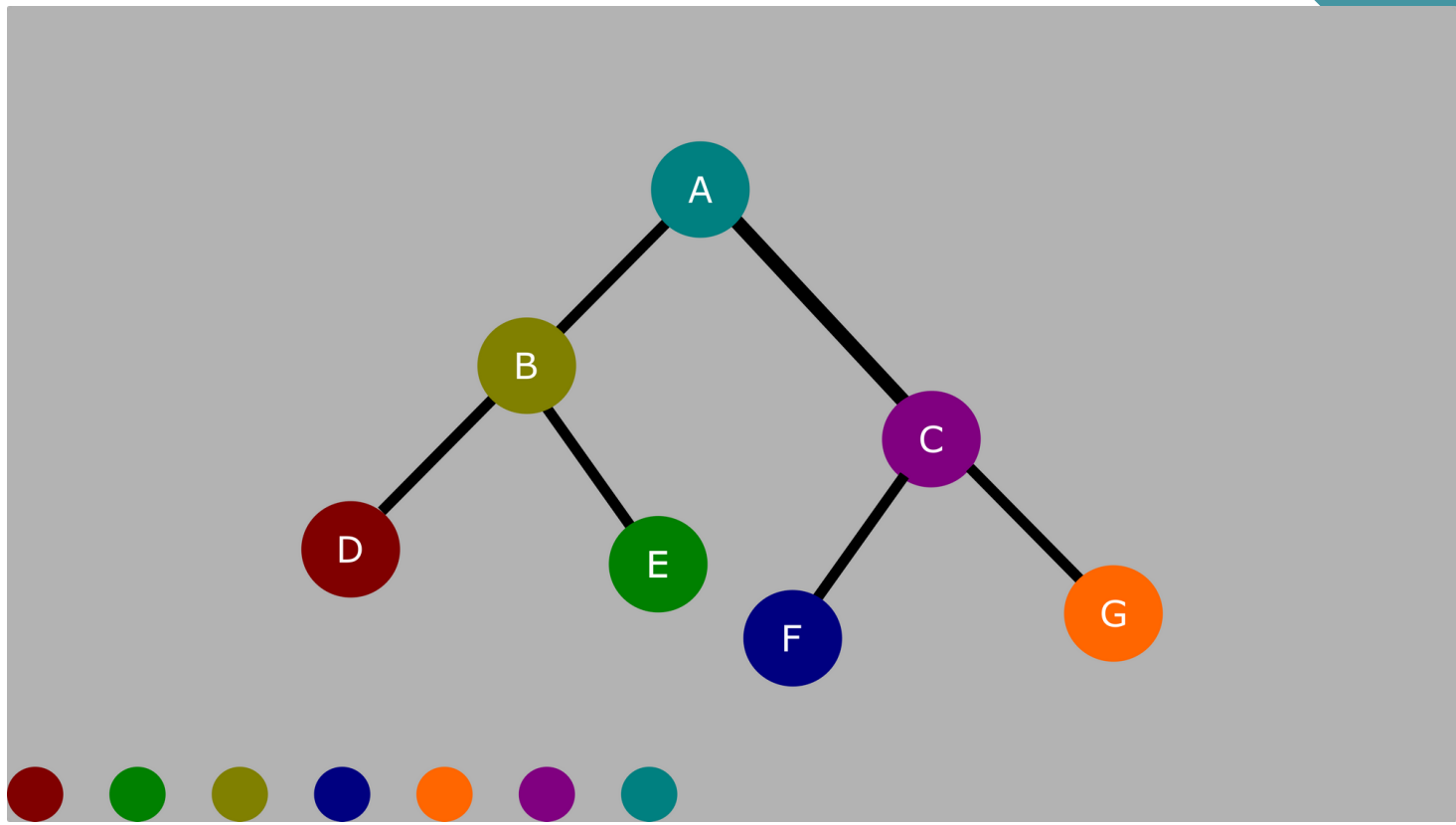


Postorder steps

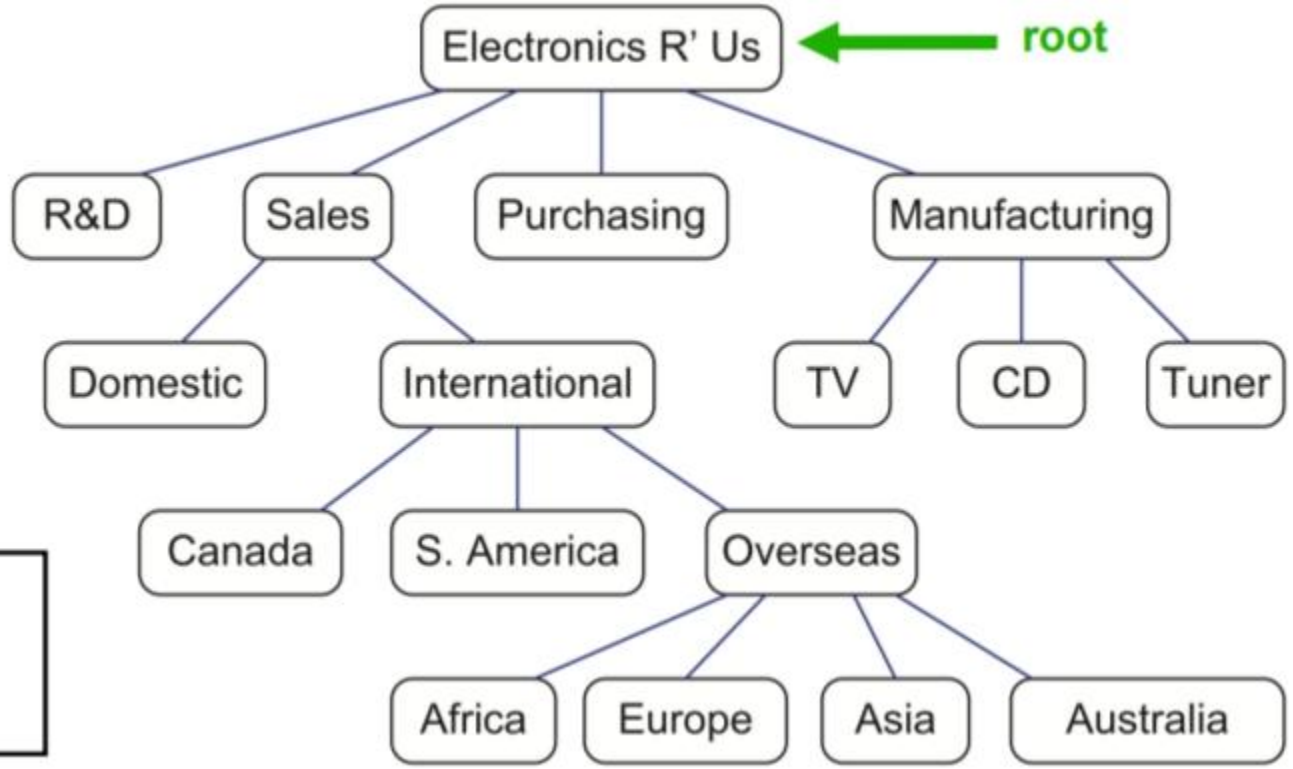
1. Go to the left subtree. (Left)
2. Go to the right subtree. (Right)
3. Visit the node. (Root)

Pseudocode (Left, Right, Root)

```
postorder(r,T):
        for each child q of r:
                postorder(q,T)
        visit node r
```

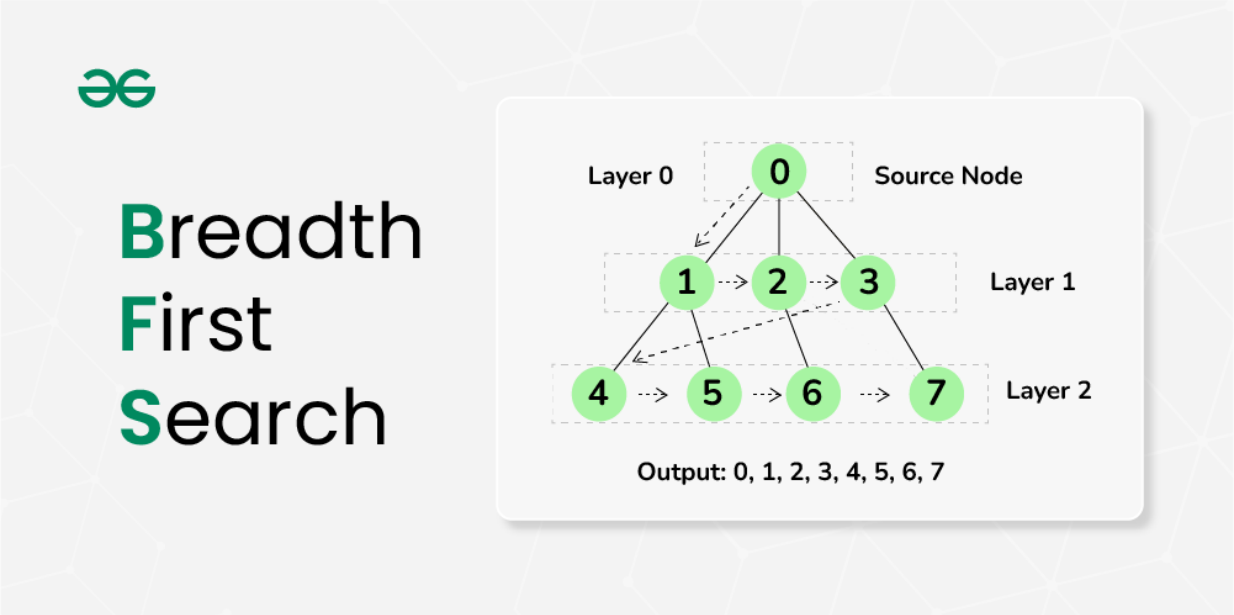


Example : Postorder traversal




```
postorder(r,T):
    for each child q of r:
      postorder(q,T)
    visit node r
```

Level order Traversal



- Level Order Traversal technique is defined as a method to traverse a Tree such that all nodes present in the same level are traversed completely before traversing the next level.
- For each node, first, the node is visited and then it's child nodes are put in a FIFO queue. Then again the first node is popped out and then it's child nodes are put in a FIFO queue and repeat until queue becomes empty.
- This visits nodes level-by-level and in left-to-right fashion at the same level.

Level order steps

1. Visit the node (Root)
2. Go to the child nodes then push all children in a queue.
3. Pop each child and push its children in the queue.

levelorder(root,T):

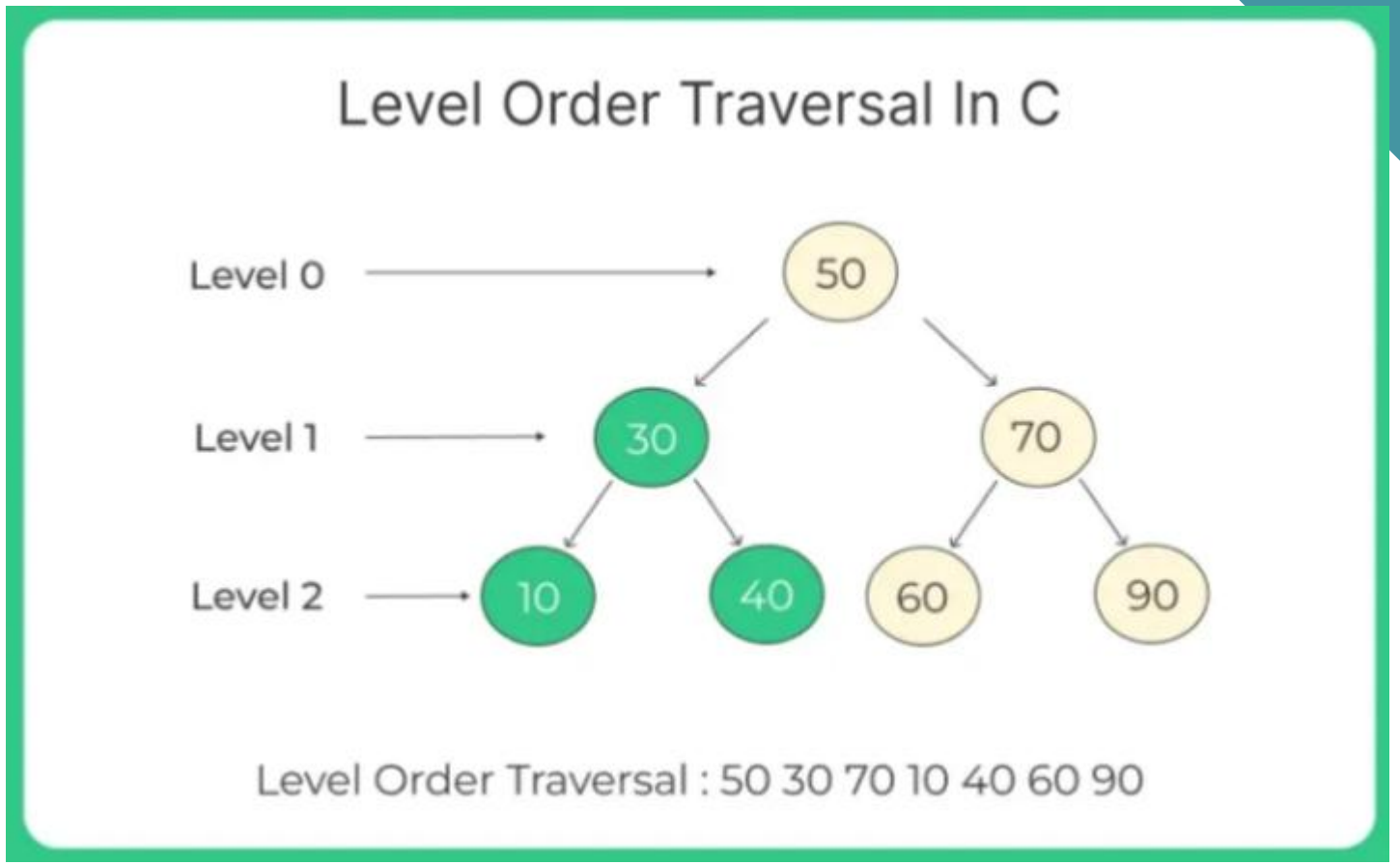
If the root is NULL, return.

Otherwise push the root in queue.

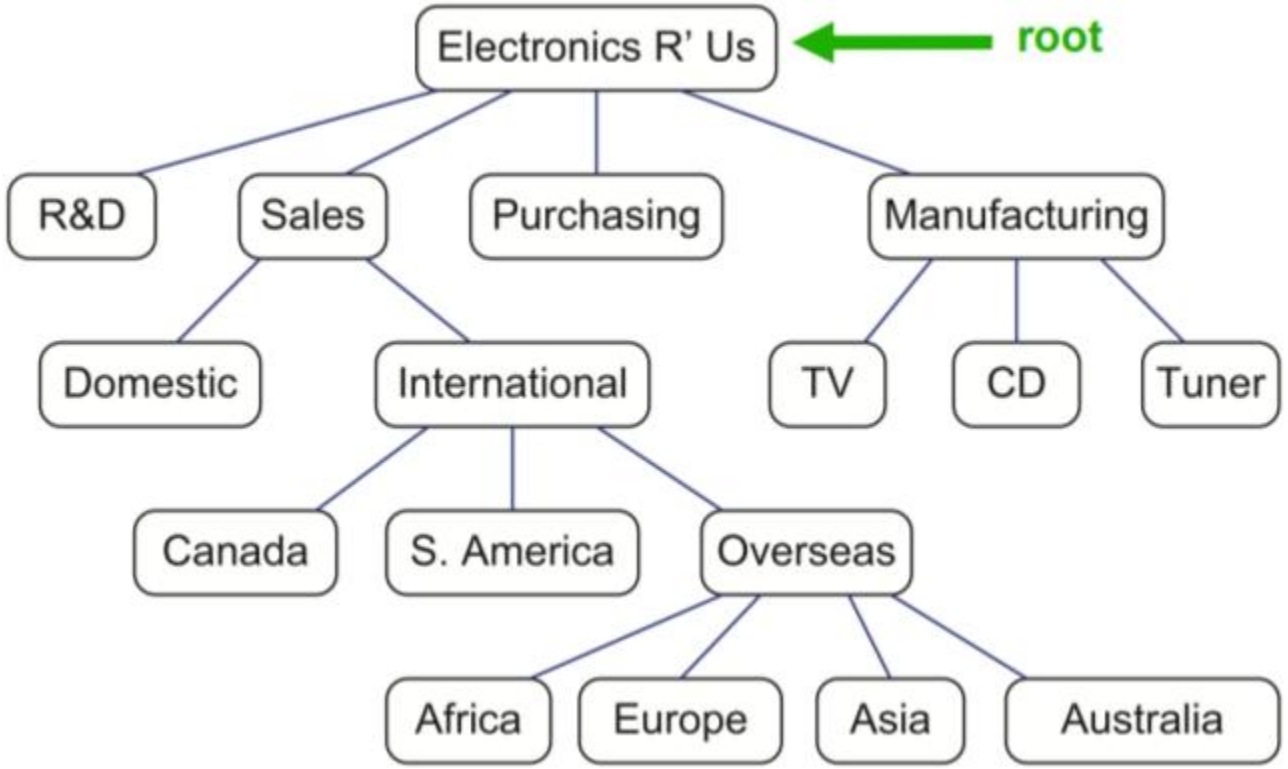
Pop the node from the queue.

Print the node's data and add its left and right child.

Repeat until the queue is empty.



Example : Levelorder traversal



Time complexity of traversals

Operations	Complexity
Preorder	$O(n)$
Postorder	$O(n)$
	Remarks: a tree of n nodes can only has *n* edges. Each traversal need to visit every node of the tree

Binary Tree

A binary tree is kind of **an ordered tree** in which every node has **at most two children**. However, if a node has just one child, the position of the child matters.

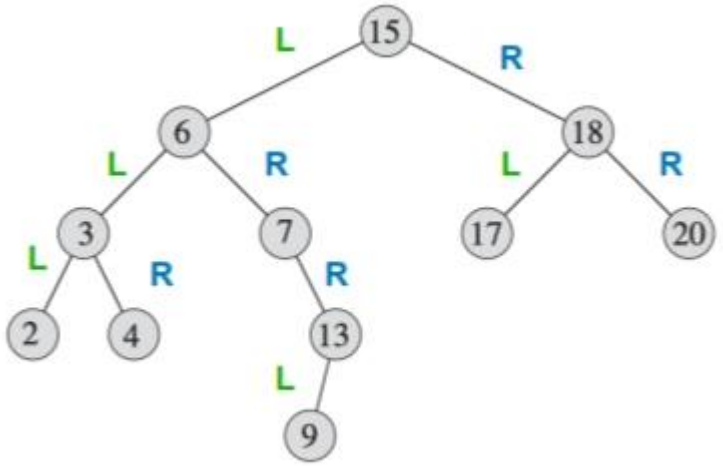
In a binary tree, every child node is labeled as being **either a left child or a right child**.

A left child precedes a right child in the ordering of children of a node

Binary Tree

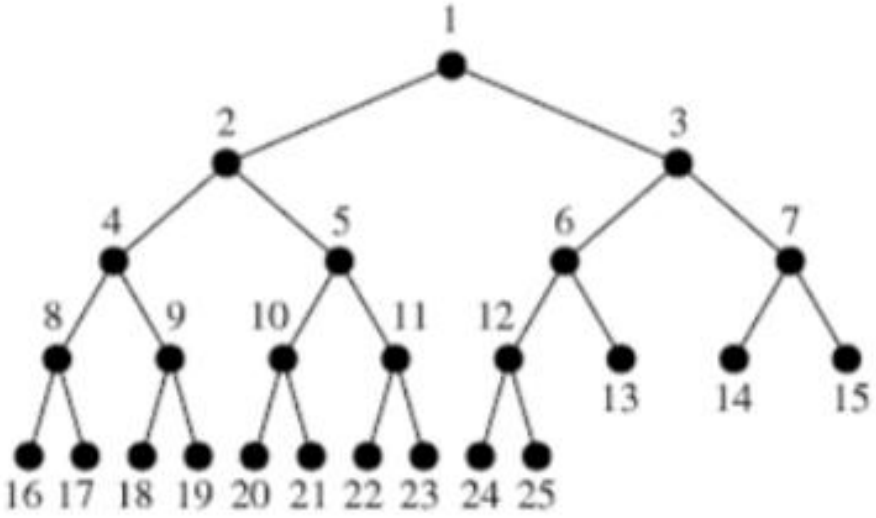
- The subtree rooted at a left or right child of an internal node is called the node's left subtree or right subtree, respectively.

- The topmost node in a binary tree is called the root, and the bottom-most nodes are called leaves. A binary tree can be visualized as a hierarchical structure with the root at the top and the leaves at the bottom.



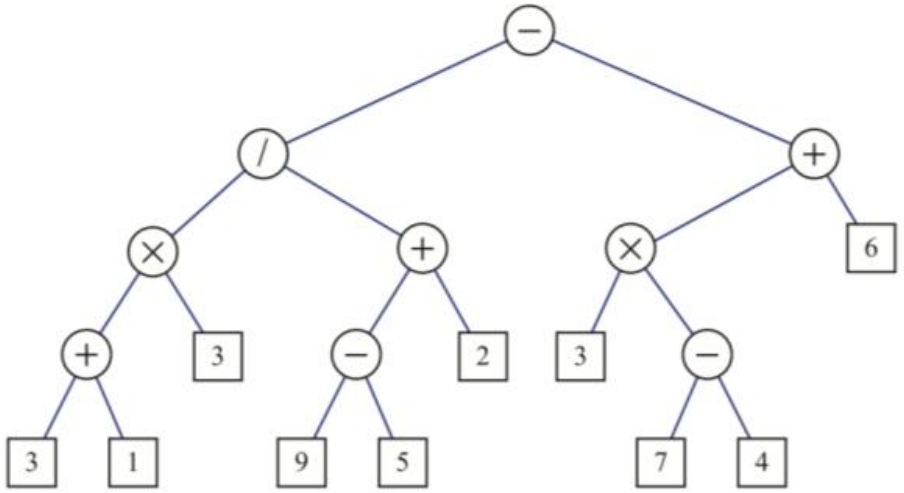
Binary Tree

- A binary tree is proper if each node has either zero or two children
  - Such trees are also referred as being **full binary trees.**
  - Thus, in a proper binary tree, every internal node has exactly two children. A binary tree that is not proper is **improper**.



An application of Binary Tree

- A binary tree can be used to represent an arithmetic expression:
  - Each node in such a tree has a value associated with it.
  - If a node is external, then its value is that of its variable or constant.
  - If a node is internal, then its value is defined by applying its operation to the values of its children.
- The above tree represents the expression ((((3+1)×3)/((9−5)+2))−((3×(7−4))+6))
- The value associated with the internal node labeled "/" is 2



Some properties of Binary Tree

The maximum number of nodes at level ‘L’ of a binary tree is 2^L.

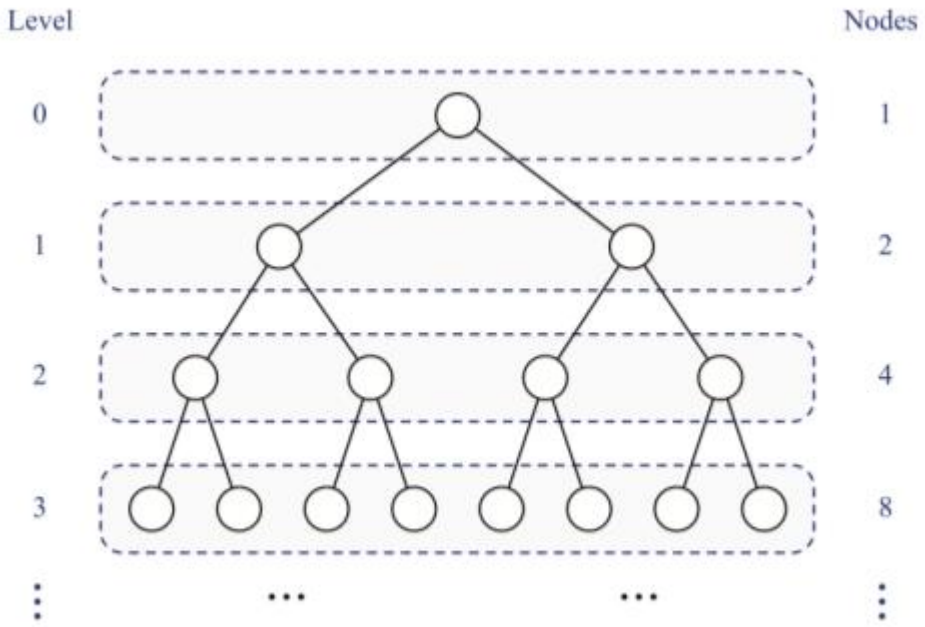
The maximum number of nodes in a binary tree of height ‘h’ is $2^h - 1$.

In a Binary Tree with N nodes, the minimum possible height or the minimum number of levels is $\text{Log}_2(N+1)$.

A Binary Tree with M leaves has at least $|\text{Log}_2 M| + 1$ levels.

The maximum number of nodes at level ‘L’ of a binary tree is 2^L.

- For root, L = 0, number of nodes = $2^0 = 1$
- Assume that the maximum number of nodes on level ‘L’ is 2L
- Since in a Binary tree every node has at most 2 children, the next level would have twice nodes, i.e. $2 * 2^L$.



The maximum number of nodes in a binary tree of height 'h' is $2^{h+1} - 1$.

- A tree has maximum nodes if all levels have maximum nodes.
- So the maximum number of nodes in a binary tree of height h is $1 + 2 + 4 + .. + 2^h = 2^{h+1}-1$.
- This is a simple geometric series with h terms and the sum of this series is $2^{h+1} - 1$.

In a Binary Tree with N nodes, the minimum possible height or the minimum number of levels is $Log_2(N+1)$.

- $N <= 2^h - 1$
- $2^h >= N+1$
- $\log_2(2^h) >= \log_2(N+1)$ (Taking log both sides)
- $h\log_2 2 >= \log_2(N+1)$
- $h >= | \log_2(N+1) |$

- So the minimum height possible is $| \log_2(N+1) |$

A Binary Tree with M leaves has at least $\lceil \log_2 M \rceil$ levels.

- $M <= 2^L$ (Property 1)
- $L = \lceil \log_2 M \rceil$
- where L is the minimum number of levels

Traversals in Binary Tree

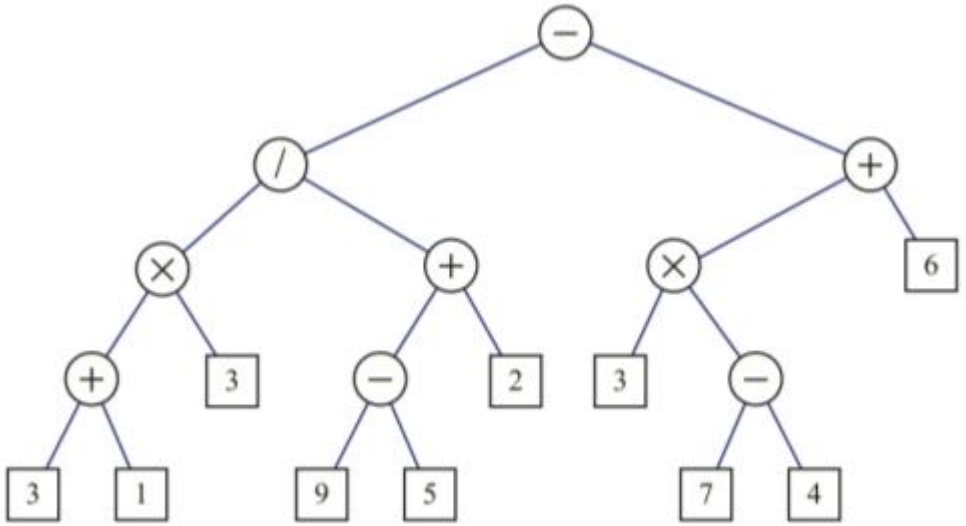
preorder(r, T): perform a preorder traversal on T

postorder(r, T): perform a postorder traversal on T

inorder(r, T): perform an inorder traversal on T

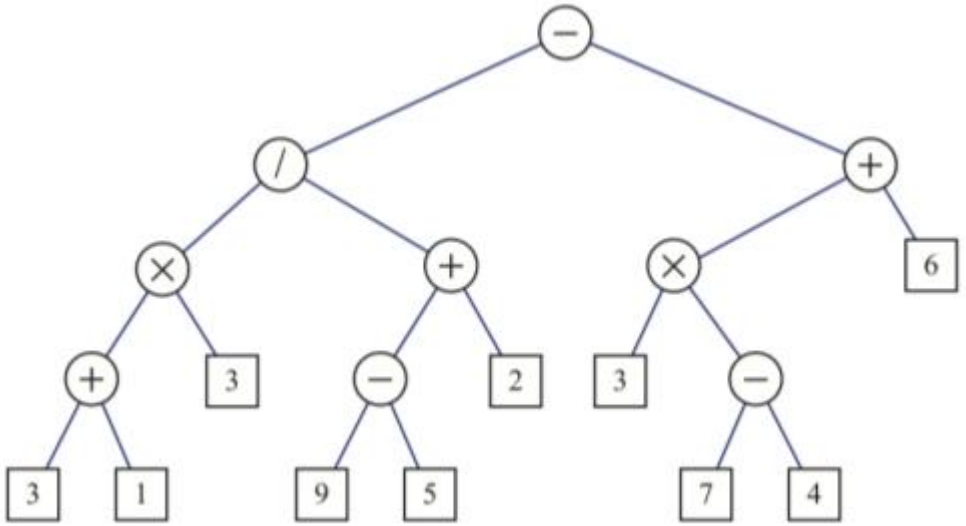
Example : Preorder traversal (Root, Left, Right)

```
preorder(r,T):
        visit node r
        if r.left:
                preorder(r.left,T)
        if r.right:
                preorder(r.right,T)
```



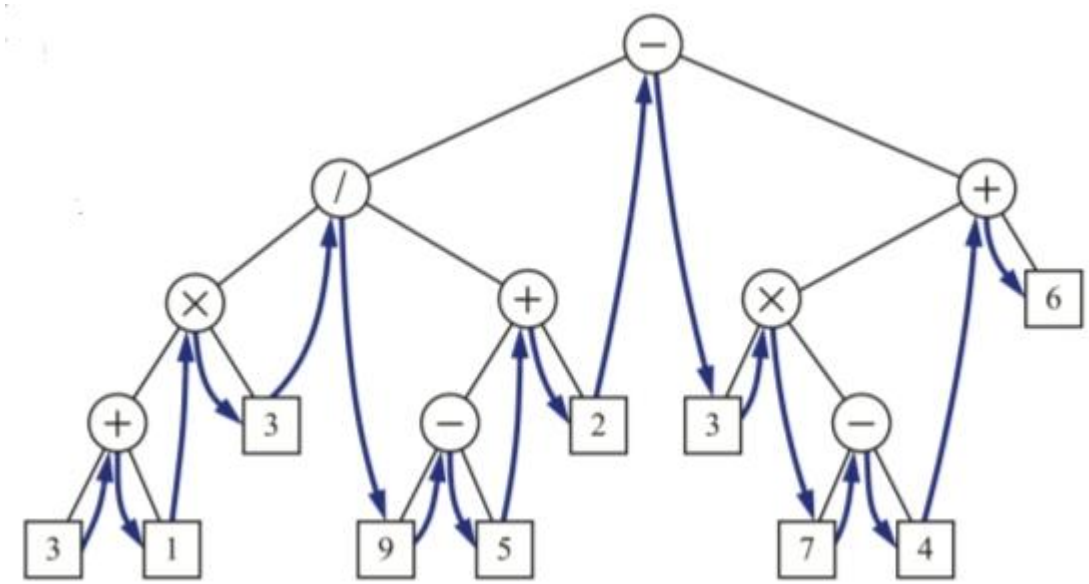
Example : Postorder traversal (Left, Right, Root)

```
postorder(r,T):
        if r.left:
                postorder(r.left,T)
        if r.right:
                postorder(r.right,T)
        visit node r
```



Example : Inorder traversal (Left, Root, Right)

```
inorder(r,T):
        if r.left:
                inorder(r.left,T)
        visit node r
        if r.right:
                inorder(r.right,T)
```



Problems on trees and binary trees

(in grader1.cs.sci.ku.ac.th)

- [Ts] Tree
- [Ts] Path of Tree
- [Ts] Largest Subtree Sum
- [BTs] Leaves of Binary Tree
- [BTs] Postorder from Preorder & Inorder